

TODO List

- ✓ Add task hierarchies for all experiments
- ✓ Review fluency and coherence [!]
- ✓ Check all internal refs
- ✓ Convert images to pdf from final year presentation
- ✓ Fix references in SOA (add to bib) [!]
- ✗ Define some named list for the contributions (C1, C2, C3...) to be recalled throughout the thesis
- ✓ To make improvement steps more gradual include single arm experiments
- ☐ Possibility: Make a teleoperated experiment with yubots



UNIVERSITÀ DEGLI STUDI
DI GENOVA

Department of Informatics, Bioengineering, Robotics and Systems
Engineering

The Thesis Title

Author's Name

Advisor: The Advisor

Co-advisor: The Coadvisor

A thesis submitted in partial fulfilment of the
PhD Program in Robotics and Autonomous Systems

Date of graduation

2019 © Author's Name

GRAAL (Genoa Robotics and Automation Laboratory)

DIBRIS DEPARTMENT, UNIVERSITY OF GENOVA

Via All'Opera Pia, 13

Genova (GE), 16145

Italia

WWW.GRAAL.DIBRIS.UNIGE.IT

First printing, March 2019

Dedicated to...

Acknowledgements

First of all I would like to thank...

Abstract

To tackle the upcoming needs of the new...

Table of Contents

Nomenclature	viii
1 Introduction	1
1.1 State of the art and innovation	2
2 Single Agent Control: Theory	3
2.1 General Definitions	5
2.1.1 Notation	5
2.2 Definitions	5
2.3 Control Objectives	6
2.3.1 Reactive Control Tasks	7
2.3.2 Non-reactive Control Tasks	8
2.4 Actions and Missions	8
2.5 Activation Functions	9
2.5.1 Action Transition	11
2.6 Task Priority Inverse Kinematics	12
Conclusions	15
List of Figures	16

Nomenclature

API	Application Programming Interface
DOF	Degrees of Freedom
DCL	Dynamic Control Layer
GUI	Graphical User Interface
HCI	Human Computer Interface
HLL	High Level Layer
iCAT	inequality Control objectives, Activations and Transitions
JCB	Joint Control Boards
KCL	Kinematic Control Layer
ROS	Robot Operating System
ROV	Remotely Operated Vehicle
RTOS	Real Time Operating System
TCP	Tool Centre Point
TPIK	Task Priority Inverse Kinematics
UDP	User Data Protocol
UVMS	Unmanned Vehicle-Manipulator System

Control tasks category identifiers

r/nr	Reactive/Non-reactive task
$=/\leq$	Equality/Inequality task
C	Physical Constraint task
S	Safety task
P	Prerequisite task
A	Action-defining task
O	Optimization task

Chapter 1

Introduction

Robots have been massively introduced in the field of manufacturing around the eighties, but ever since they have been mainly used in very fixed and structured environments and with no or rare interaction of robots with human workers. Nowadays new kind of robotic manipulators, including dual arm setups and mobile manipulators, are being rolled out from bigger producers such as KUKA, ABB, Universal Robots, Rethink Robotics and more, featuring lightweight and compliant structures that can be exploited to tackle the upcoming needs of the new Industry 4.0 paradigm where cyber-physical systems communicate and cooperate with each other and humans in real time. These trends have been especially identified, in the European Multi-Annual Roadmap for robotics, as innovations with high potential impact on market domains [?] and covered in a series EU H2020 calls, such as the recent ICT (Information and Communication Technology) one on *Advanced robot capabilities* research [?]. Towards that end, the control of the robot must be able to guarantee a high level of flexibility in order to accomplish a variety of tasks that go beyond the usual, standardized, robotic cell ones, which use to take place within a very structured environment. The basic idea is that the robot should autonomously take care of all the control objectives related to the safety of the system, interaction with the environment, as well as those that are a prerequisite to accomplish the required task. Achieving these objectives would indeed grant the robot a higher level of flexibility, simplifying the higher levels that deal with planning and task sequencing.

This PhD work is in fact framed within the Italian technology cluster *Fabbrica Intelligente* (i.e. Smart Manufacturing). In particular, as seen in figure 1.1, in the Italian “*Research and Innovation Roadmap*” [?], a series of the key enabling tech-

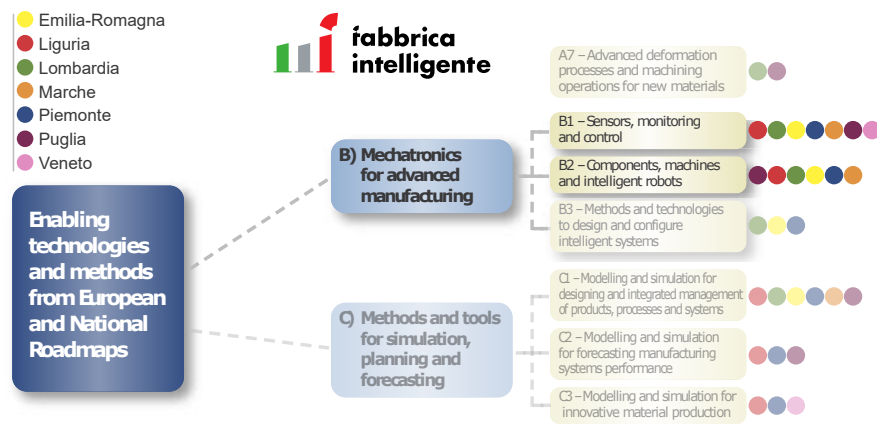


Figure 1.1: A focused extract from the Italian “*Research and Innovation Roadmap*” [?] for smart manufacturing, highlighting the scope in which this PhD work is framed.

nologies have been identified in the area of “Mechatronics for advanced manufacturing”, where the Liguria region in which our university resides, is involved in the development of *Sensors, monitoring and control* and *Components, machines and intelligent robots*.

1.1 State of the art and innovation

Redundant robotic systems are becoming increasingly more...

Chapter 2

Single Agent Control: Theory

As anticipated in the introduction the control framework presented in this thesis is based on the cascade of blocks shown in Fig. 2.1. In particular, the architecture is constituted by three main general blocks:

1. The Mission Manager is in charge of supervising the execution of the current *mission*, and generates the corresponding *actions* to be executed by the Kinematic Control Layer. As it will be explained later in section 2.4, an action is any prioritized list of control objectives to be concurrently achieved, and a mission is a sequence (or graph) of actions.
2. The Kinematic Control Layer (KCL) implements the proposed task priority control framework, and is in charge of reactively accomplishing the *control objectives* that make up the current action to be executed, generating the desired system velocity vector.
3. The Dynamic Control Layer (DCL) tracks the desired system velocity vector by generating appropriate force/torques references for the vehicle and the manipulator.

The thesis analysis focuses on the Kinematic Control Layer, since it is the one implementing the proposed task priority approach. The interfaces with the higher level (Mission Manager) and the lower one (DCL) are highlighted whenever relevant. By the definition we have given for the Mission Manager follows that it will be application dependent. In general it is the higher level layer in a given software architecture. Therefore we may also refer to it as the HLL (High Level Layer).

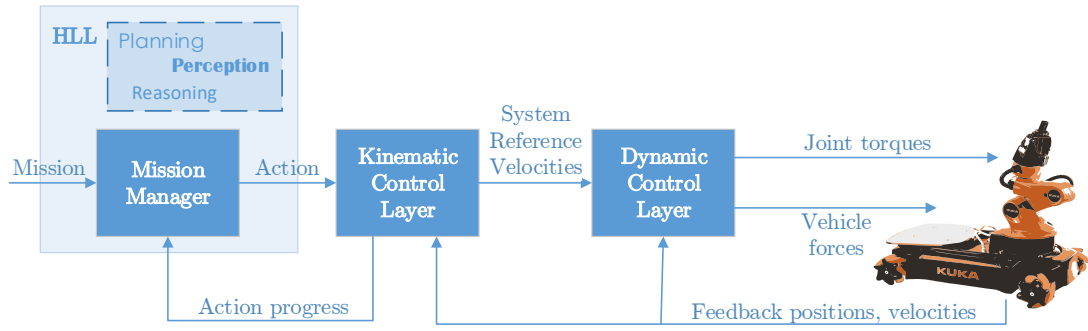


Figure 2.1: The overall architecture: the Kinematic Control Layer is the one implementing the proposed task priority approach, executing the current action scheduled by a Mission Manager; the output of the Kinematic Control Layer are the system velocities, tracked by the underlying Dynamic Control Layer.

The unifying KCL control architecture presented can be applied both in the case of partial (assisted) teleoperation and fully autonomous operation. The main features of this architecture can be summarized as:

- F_1 inequality control objectives are handled without over-constraining the system, allowing them to be placed at higher priority whenever necessary, e.g. safety control objectives can have high priority (section 2.5);
- F_2 switching between actions (an action is a set of prioritized control tasks) can be done easily, and allows the higher planning levels to interface with the control one in terms of such building blocks, which cover a great deal of underlying control details (section 2.5.1).
- F_3 the presence of vehicle underactuations can be managed, to the best extent possible, with the addition of a suitable highest priority non-reactive control task (section ??);
- F_4 vehicle inaccuracies in tracking the desired velocity are not propagated to the end-effector thanks to a vehicle-arm coordination scheme, based on two parallel optimizations (section ??);
- F_5 the vehicle-arm coordination scheme naturally allows a multi rate control of the two subsystems (section ??);

2.1 General Definitions

Before starting the discussion we first introduce some notation in the next subsection. Then some definitions are given in the successive subsection, while some preliminaries on the regularized pseudo inverses are given in the last one.

2.1.1 Notation

Vectors and matrices are expressed with a bold face character, such as $\mathbf{M}$, whereas scalar values are represented with a normal font such as γ . Given a matrix $\mathbf{M}$:

- $M_{(i,j)}$ indicates the element of $\mathbf{M}$ at the i -th row and j -th column;
- $\mathbf{M}^\#$ is the exact generalized pseudo-inverse, i.e. the pseudo inverse of $\mathbf{M}$ performed without any regularizations.

Further, less used notation is introduced as needed.

2.2 Definitions

Let us introduce some definitions that will be used thoroughly in this paper. Here we will consider the general case of a floating dual-arm manipulator, which encompasses all the type of robots, then every other system considered successively will lie in a subspace derived by an appropriate downgrade of this general case. Let us define:

- The system configuration vector $\mathbf{c} \in \mathbb{R}^n$ of a generic mobile manipulator as

$$\mathbf{c} \triangleq \begin{bmatrix} \boldsymbol{\eta} \\ \mathbf{q}_L \\ \mathbf{q}_R \end{bmatrix}, \quad (2.1)$$

where $\boldsymbol{\eta} \in \mathbb{R}^m$ is the vehicle *generalized coordinate position vector* and $\mathbf{q}_L \in \mathbb{R}^l$ and $\mathbf{q}_R \in \mathbb{R}^r$ are the left and right arm configuration vectors, supposing that that the left arm has l DOF and the right one has r DOF. In particular $\boldsymbol{\eta}$ is the stacked vector of the position vector $\boldsymbol{\eta}_1$, with components on the inertial frame $\langle w \rangle$, and the orientation vector $\boldsymbol{\eta}_2$, the latter expressed in terms of the three angles yaw, pitch and roll (applied in this sequence) [?]. From the above definitions it results $n = l + r + m$;

- The system velocity vector $\dot{\mathbf{y}} \in \mathbb{R}^n$ of the mobile manipulator as

$$\dot{\mathbf{y}} \triangleq \begin{bmatrix} \mathbf{v} \\ \dot{\mathbf{q}}_L \\ \dot{\mathbf{q}}_R \end{bmatrix}, \quad (2.2)$$

where $\mathbf{v} \in \mathbb{R}^m$ is the stacked vector of the vehicle linear velocity vector $\mathbf{v}_1$ and the vehicle angular velocity vector $\mathbf{v}_2$, both with components on the vehicle frame $\langle \mathbf{v} \rangle$. While $\dot{\mathbf{q}}_L \in \mathbb{R}^l$ and $\dot{\mathbf{q}}_R \in \mathbb{R}^r$ are the left and right arm joint velocities. For the time being, this work will be developed with the assumption of a vehicle fully actuated in its 6 degrees of freedom, therefore $m = 6$, and the system velocity vector will coincide with the control vector. Section ?? will show how to relax this hypothesis.

2.3 Control Objectives

Let us start the discussion by considering what the robotic system needs to achieve. The goals of the system might include, for example, maintaining a certain distance from another vehicle, reaching a waypoint, or avoiding the arm joint limits. Mathematically speaking, let us consider a scalar variable $x(\mathbf{c})$, i.e. a variable dependent on the current configuration of the robot and the environment. Then, for this variable, two broad classes of objectives can be defined:

1. The requirement, for $t \rightarrow \infty$, that $x(\mathbf{c}) = x_0$ is called a *scalar equality control objective*.
2. The requirement, for $t \rightarrow \infty$, that $x(\mathbf{c}) < x_M$ or $x(\mathbf{c}) > x_m$ is called a *scalar inequality control objective*.

The mathematical symbols ‘=’ and ‘ $\leq$ ’ will be used throughout the thesis to identify equality and inequality scalar objectives respectively.

The use of scalar objectives is to simplify the presentation of the approach. However, let us remark that it is not a limitation of the approach. In fact, consider a vector $\mathbf{h} \in \mathbb{R}^m$, and consider m different scalar objectives, one for each of its components. Then, all together the m objectives allow to define a requirement for the whole vector $\mathbf{h}$, for example having some of its components within some ranges, while the others attaining certain values.

Other than the specific class, the control objectives can be divided in different categories, depending on their purpose and, as it will be clear later, on their priority. In fact, control objectives can be classified as:

- C physical **constraints** objectives, i.e. interacting with the environment;
- S system **safety** objectives, e.g. avoiding joint limits or obstacles;
- P objectives that are a **prerequisite** for accomplishing the mission, e.g. maintaining the manipulated object in the camera angle of view;
- A **action** defining objectives, i.e. what the system really needs to execute to accomplish the current user defined action;
- O **optimization** objectives, i.e. objectives that do not influence the mission, but allow to choose between multiple solutions, if they exists.

The capitalized letters associated with the class types will be used throughout the thesis to identify a specific type of objective (the Nomenclature section at the beginning can be used as a reference).

2.3.1 Reactive Control Tasks

To each scalar control objectives there is always associated a *feedback reference rate* $\dot{\hat{x}}$, whose role is to drive the associated variable $x(c)$ toward an arbitrary point x^* , located inside the interval where the objective is satisfied. For instance, an example of feedback reference rate is the following one:

$$\dot{\hat{x}}(x) \triangleq \gamma(x^* - x), \gamma > 0, \quad (2.3)$$

where γ is a positive gain proportional to the desired convergence rate for the considered variable.

For each of the considered variables $x(c)$, the Jacobian relationship

$$\dot{x} = J\dot{y} \quad (2.4)$$

links the rate of change of the variable itself with the system velocity vector $\dot{y}$ through the vector $J \in \mathbb{R}^n$. So, having the actual $\dot{x}$ as much as possible equal to the desired $\dot{\hat{x}}$ is termed a *reactive control task*. The letter ‘r’ will identify reactive tasks throughout the thesis.

2.3.2 Non-reactive Control Tasks

In the previous sections, the triple control objective, reactive control task and activation function has been defined. However, there are cases where a control task is defined directly in a certain task velocity space, without having an associated control objective. As an example, consider the case where a human operator wants to control the end-effector by generating velocity references through a joystick. In such a case, there is no control objective associated with the control task, because there is not a position to reach. In fact, the reference rate is generated by the user, rather than being the output of a feedback control loop as in (2.3). The letters pair ‘nr’ will identify non-reactive tasks throughout the thesis.

2.4 Actions and Missions

In the terminology of this work, an *action* is a prioritized list of control objectives and associated reactive tasks, with the possible addition of any non-reactive tasks, to be *concurrently* managed. For example, for a robotic manipulator, a manipulation action could be represented by following list of objectives

- $\mathbf{o}_1)$ $[\mathbf{r}, \leq, S]$ Arm joints limits
- $\mathbf{o}_2)$ $[\mathbf{r}, \leq, S]$ Arm manipulability
- $\mathbf{o}_3)$ $[\mathbf{r}, =, A]$ Tool frame position control
- $\mathbf{o}_4)$ $[\mathbf{r}, =, A]$ Tool frame orientation control
- $\mathbf{o}_5)$ $[\mathbf{r}, \leq, 0]$ Arm preferred shape

where it is natural to see *safety* objectives (S) such as joint limits at the highest priority. It is followed by the manipulability objective that allows for a better execution of the overall position objective, since it avoids the problems of high joint velocities occurring near kinematic singularities. Finally, an optimization objective such as the preferred shape can be defined, however its priority will be lower than the position control, to avoid over-constraining the system to a specific configuration. See Section 2.3 for more insight on task types. A more in-depth explanation for the single tasks will be introduced in the application section, every time a new control objective is introduced.

Further examples will be given in Chapter ???. For the time being, the following aspects can be highlighted:

- In principle, within an action, there could be an arbitrary number of prioritized objectives, organized in different ways. However, following the categories of objectives presented in section 2.3, usually the main difference between actions will be the objective(s) needed to achieve its particular ultimate goal and possibly some other prerequisite ones, while the safety tasks will be the same.
- Actions can be temporally sequenced accordingly with a *mission* plan represented by a decisional action graph, having the actions as nodes and the arcs as logic decision alternatives, with just a few parameters relevant to the action ultimate-goal only.
- The aforementioned two points allow to highlight how the interface of the KCL with the Mission Manager should therefore result simplified, allowing an easier implementation of the latter.
- Moreover, transitions from an action to another one located at the end of a selected arc, should be smoothly activated.

2.5 Activation Functions

For every control objective and relative reactive control task, there is always an associated activation function to be considered. The role of the activation function, as the name implies, is to specify whenever the control objective is relevant for the system. For example:

- Maintaining the object to be grasped within the camera system cone of vision is relevant only when the object is close to the boundaries, since there is no need to have it *exactly* centred;
- Maintaining a joint away from its limit is relevant only in the vicinity of the limit itself, but it *should not overconstrain* the system when the joint is sufficiently far away.

Motivated by the above considerations, let us define an activation function so composed:

$$a(x) = a^i(x), \quad (2.5)$$

where $a^i(x) \in [0, 1]$ is continuous sigmoidal function of the objective variable x , which assumes zero values within the validity region of the associated objective. For example, assuming an objective of the class $x(c) > x_{min}$:

$$a^i(x) = \begin{cases} 1, & x(c) < x_{min} \\ s(x), & x_{min} < x(c) < x_{min} + \Delta \\ 0, & x(c) > x_{min} + \Delta \end{cases} \quad (2.6)$$

where $s(x)$ is any continuous function joining the two extrema, and Δ is a value that creates a buffer zone, where the inequality is already satisfied, but the activation value is still greater than zero, preventing chattering problems around the inequality control objective threshold x_{min} . An example of sigmoidal activation function is reported in Fig. 2.2b. Throughout the thesis we will make use of the sigmoidal function so it's useful to define a compact notation to identify different types of sigmoids. Having $[x_{min}, x_{MAX}]$ the two minimum and maximum extrema for the sigmoid curve on the x axis, and $[0, 1]$ the two extrema for the sigmoid curve on the y axis (the activation value), we can define:

- An increasing sigmoidal activation function as

$$\sigma_{\uparrow}(x_{min}, x_{MAX}) \quad (2.7)$$

- A decreasing sigmoidal activation function as

$$\sigma_{\downarrow}(x_{min}, x_{MAX}) \quad (2.8)$$

In general, for multidimensional control objective the activation function for a particular task k will be a matrix of this type:

$$A_k = \begin{bmatrix} a_{k,1} & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & a_{k,l} \end{bmatrix}, \quad (2.9)$$

where here $a_{k,1}$ is the i -th element on the trace of A for the k -th task.

For non-reactive control tasks, since the variable $x(c)$ is not defined, the activation function (2.5) reduces to be $a(x) = a^i(x) \equiv 1$ as in Figure 2.2a. From now on, the distinction between reactive and non-reactive control tasks will be dropped, and the generic term control task will be used, unless otherwise specified.

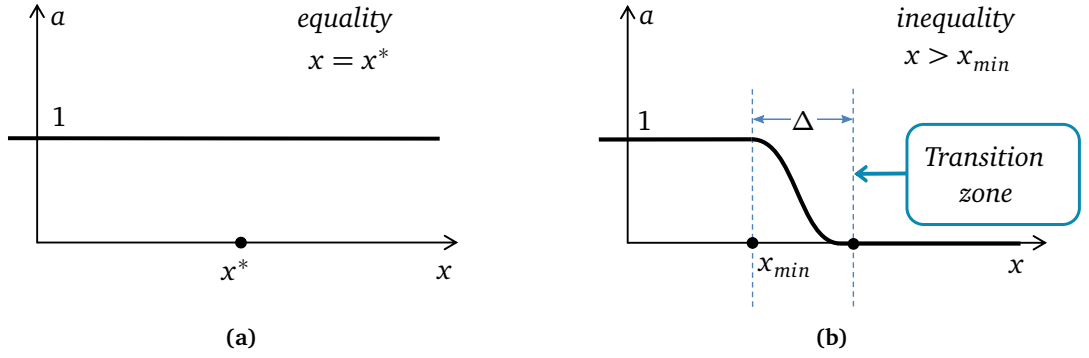


Figure 2.2: Comparison of task activations for equality and inequality control objectives. In figure 2.2b and example of $\sigma_{\downarrow}(x_{min}, x_{min} + \Delta)$ decreasing sigmoidal function.

2.5.1 Action Transition

The activation and deactivation of a task, apart from avoiding unnecessary system constraints, can serve another important purpose. In fact, it can be exploited for providing a smooth transition between actions, so to satisfy the requirements highlighted in 2.4. Let us consider the following two actions $\mathcal{A}_1$ and $\mathcal{A}_2$, composed by objectives abstractly labelled with alphabetic letters, where $A \prec B$ denotes that A has higher priority than B :

$$\mathcal{A}_1 : A \prec B, C, D$$

$$\mathcal{A}_2 : A \prec D \prec C, E$$

where A, C, D are in common, but with D at a different priority ordering w.r.t. C . Now consider the following merged list:

$$\mathcal{U} : A \prec D_1 \prec B, C, D_2, E;$$

where D_1 and D_2 represent the same objective D , but with a different priority. It is clear that, through insertion/deletion of some of the entries, the two original lists can be reconstructed. For example, $\mathcal{A}_1$ can be obtained by removing D_1 and E from the unified list. Conversely, $\mathcal{A}_2$ can be obtained by removing the control objectives D_2 and E .

To make possible this transitions when switching between actions, the activation function (2.5) is modified to become:

$$a(x, \mathbf{p}) = a^i(x) a^p(\mathbf{p}), \quad (2.10)$$

where $a^p(\mathbf{p}) \in [0, 1]$ is a continuous sigmoidal function of a vector of parameters $\mathbf{p}$ external to the control task itself. In particular, the $a^p(\mathbf{p})$ can be conveniently parametrized by the previous and current action being executed, and the

time elapsed in the current action, to obtain the desired activation/deactivation smooth transition. Of course, this new activation function $a^p(\mathbf{p})$ applies to both to reactive and non-reactive control tasks since it does not depend on the variable $x(\mathbf{c})$. Having N tasks, identified by their k index, and two successive actions $\mathcal{A}_1$ and $\mathcal{A}_2$, we can define the following algorithm for smoothly transition between actions:

Algorithm 1 Tasks transition algorithm

```

1: for  $k \leftarrow 1$  to  $N$  do
2:   if  $\text{task}_k \in \mathcal{A}_1 \wedge \text{task}_k \in \mathcal{A}_2$  then
3:      $a^p(\mathbf{p}) = 1$ 
4:   end if
5:   if  $\text{task}_k \in \mathcal{A}_1 \wedge \text{task}_k \notin \mathcal{A}_2$  then
6:      $a^p(\mathbf{p}) = \text{decreasingSigmoid}(t)$ 
7:   end if
8:   if  $\text{task}_k \notin \mathcal{A}_1 \wedge \text{task}_k \in \mathcal{A}_2$  then
9:      $a^p(\mathbf{p}) = \text{increasingSigmoid}(t)$ 
10:  end if
11: end for
    
```

Once all the actions that the system can execute have been defined, the unified list can be constructed. Many safety tasks will be common to all actions, and they will be at the same (high) priority level in each of them, therefore for such tasks it will result that $a^p(\mathbf{p}) \equiv 1$. Conversely, $a^p(\mathbf{p})$ will be mostly exploited to activate/deactivate action-defining objectives and prerequisite ones, calculated as in Algorithm 1.

2.6 Task Priority Inverse Kinematics

Given an action $\mathcal{A}$, for each priority level k , the following matrices and vectors are consequently defined:

- $\dot{\mathbf{x}}_k \triangleq [\dot{x}_{1,k} \ \cdots \ \dot{x}_{m,k}]^T$ is the stacked vector of all the reference rates of the scalar control tasks;
- $\mathbf{J}_k$ is the Jacobian relationship expressing the current rate-of-change of the k -th task vector $[\dot{x}_{1,k} \ \cdots \ \dot{x}_{m,k}]^T$ w.r.t. the system velocity vector $\dot{\mathbf{y}}$, simply resulting from the stacking of the row vector ones in (2.4);
- $\mathbf{A}_k \triangleq \text{diag}(a_{1,k}, \dots, a_{m,k})$ is the diagonal matrix of all the activation functions described in (2.10).

Let us remark how inequalities introduced in section 2.3 are objectives to be eventually reached that, with the adoption of the reference law (2.3), are transformed in control tasks, i.e. velocity tracking minimization problems. However, as already mentioned, to avoid overconstraining the system, they must be deactivated whenever the inequality is satisfied.

To the aim of finding the system velocity reference vector $\dot{\mathbf{y}}$ that satisfies the above expressed priority requirements between the different objectives, a straightforward idea is to employ the diagonal matrix $\mathbf{A}_k$ as a weight in the minimization process to deactivate tasks that are not anymore relevant, thus leading to the following sequence of nested minimization problems, corresponding to the so called Task Priority Inverse Kinematics (TPIK):

$$S_k \triangleq \left\{ \arg \min_{\dot{\mathbf{y}} \in S_{k-1}} \left\| \mathbf{A}_k (\dot{\mathbf{x}}_k - \mathbf{J}_k \dot{\mathbf{y}}) \right\|^2 \right\}, \quad k = 1, 2, \dots, N, \quad (2.11)$$

where S_{k-1} is the manifold of solutions of all the previous tasks in the hierarchy, $S_0 \triangleq \mathbb{R}^n$, and N is the total number of priority levels. Unfortunately, the task priority implementation as suggested above turns out to be affected by discontinuities in $\dot{\mathbf{y}}$ when $\mathbf{A}_k$ is varying some of its entries (i.e. within the so called transition zones) due to the fact that the resulting weighted pseudo-inverse $(\mathbf{A}_k \mathbf{J}_k)^\# \mathbf{A}_k$ is actually invariant to the weights on the rows of $\mathbf{J}_k$ that are linearly independent, as shown in [?].

In order to cope with these invariance problems, a specialized *task oriented regularization* has been introduced in the minimization problem, as explained in [?]. Such a specialized regularization, coupled with the classical singular value oriented one to deal with singularities of the involved matrices, was the novel idea of the proposed new pseudo inversion framework.

The adopted specialized regularization will be now briefly outlined, by now making reference to the following new form of TPIK problem itself

$$S_k \triangleq \left\{ \arg \text{R-} \min_{\dot{\mathbf{y}} \in S_{k-1}} \left\| \mathbf{A}_k (\dot{\mathbf{x}}_k - \mathbf{J}_k \dot{\mathbf{y}}) \right\|^2 \right\}, \quad k = 1, 2, \dots, N, \quad (2.12)$$

where the notation R-min is introduced for underlining that each minimization is performed via the specialized form of [?].

Within such a specialized framework (named *iCAT: inequality Constraints And Task transitions*) the above TPIK problem (2.12) results in the following algorithm, to be initialized with

$$\boldsymbol{\rho}_0 = \mathbf{0}, \quad \mathbf{Q}_0 = \mathbf{I}, \quad (2.13)$$

and then, for $k = 1, \dots, N$

$$\begin{aligned} W_k &= J_k Q_{k-1} (J_k Q_{k-1})^{\#, A_k, Q_{k-1}}, \\ Q_k &= Q_{k-1} (I - (J_k Q_{k-1})^{\#, A_k, I} J_k Q_{k-1}), \\ \rho_k &= \rho_{k-1} + \text{Sat} \left(Q_{k-1} (J_k Q_{k-1})^{\#, A_k, I} W_k (\dot{\mathbf{x}}_k - J_k \rho_{k-1}) \right), \end{aligned} \quad (2.14)$$

where

- the special pseudo inverse operator $(\cdot)^{\#, A, Q}$ is strictly needed to cope the invariance problems mentioned above. Its definition is reported in [?] and will be omitted here.
- ρ_k is the vector of the system control variables, which results constructed at the k -th minimization stage, through the last equation in (2.14);
- $(\dot{\mathbf{x}}_k - J_k \rho_{k-1})$ is the k -th task reference rate minus the task space contribution of the $k-1$ control vector ρ_{k-1} , i.e. the influence exerted on the current k -th task by the presence of all the higher priority tasks;
- Q_{k-1} is a $n \times n$ matrix that takes into account the control directions (totally or partially) used within all the higher priority tasks;
- W_k is a $m \times m$ matrix, where m is the row-dimension of the control task at the current priority level, which is suitably structured as indicated to prevent discontinuities in the control variables occurring between different priority levels;
- The function $\text{Sat}(\cdot)$ implements the management of control variable saturations, accordingly with what suggested in [?].

At the end of the above iterative process, the final system velocity vector is therefore $\dot{\mathbf{y}} = \rho_N$. The interested reader can find all the relevant details of this procedure in [?], where a comparison with other task priority approaches, as for instance those appearing in [?, ?, ?], is given.

Conclusions

In this thesis a modular and expandable control framework for mobile manipulators has been presented.

Firstly, in the *Introduction* chapter the framing and the rationale behind this work have been explained. The main reasoning lays its basis...

Future Work

Even with all the effort one can put in, there's always room for improvement. For what concerns...

List of Figures

1.1	A focused extract from the Italian “ <i>Research and Innovation Roadmap</i> ” [?] for smart manufacturing, highlighting the scope in which this PhD work is framed.	2
2.1	The overall architecture: the Kinematic Control Layer is the one implementing the proposed task priority approach, executing the current action scheduled by a Mission Manager; the output of the Kinematic Control Layer are the system velocities, tracked by the underlying Dynamic Control Layer.	4
2.2	Comparison of task activations for equality and inequality control objectives. In figure 2.2b and example of $\sigma_{\downarrow}(x_{min}, x_{min} + \Delta)$ decreasing sigmoidal function.	11